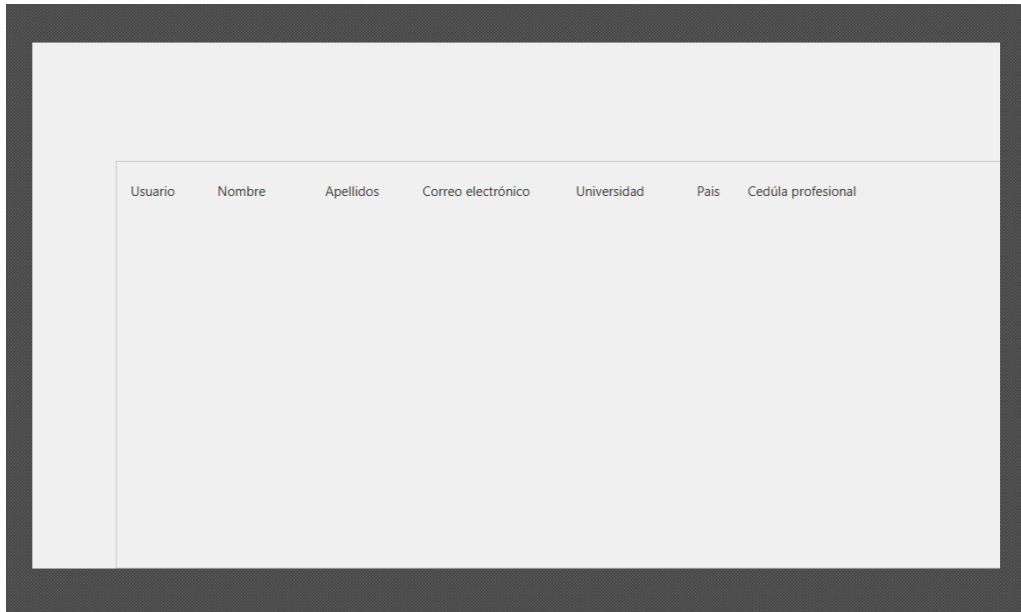


Avance 26/04/2024

Fernando Martínez Ramírez

1. Ventana: Gestionar cuenta.

Permite al administrador aceptar o rechazar una cuenta que se encuentre en estado pendiente.



2. Item: CuentaltemControlador

Por cada cuenta registrada que se encuentre en estado pendiente. crea un nuevo item "CuentaltemControlador"

"CuentaltemControlador" cuenta con un metodo setLabel, este metodo se encarga de mostrar los datos correspondientes en las etiquetas.

```
1 usage  FerRMZ
public void setLabel (Cuenta cuenta) {
    lbUsuario.setText(cuenta.getNombreUsuario());
    Academico academico = getAcademico(cuenta.getIdPersona());
    lbNombre.setText(academico.getNombre());
    lbApellidos.setText(academico.getApellidoPaterno() + " " + academico.getApellidoMaterno());
    lbCorreo.setText(academico.getCorreoElectronico());
    lbCedula.setText(academico.getCedulaProfesional());
    Universidad universidad = getUniversidadPorId(academico.getIdUniversidad());
    lbUniversidad.setText(universidad.getNombre());
    Pais pais = getPaisPorId(universidad.getIdPais());
    lbPais.setText(pais.getNombre());
}
```

3. Controlador: gestionCuentaControlador

En el metodo inicializable obtiene las cuentas en estado pendiente y crea un objeto de tipo cuentaltemControlador por cada cuenta en estado pendiente.

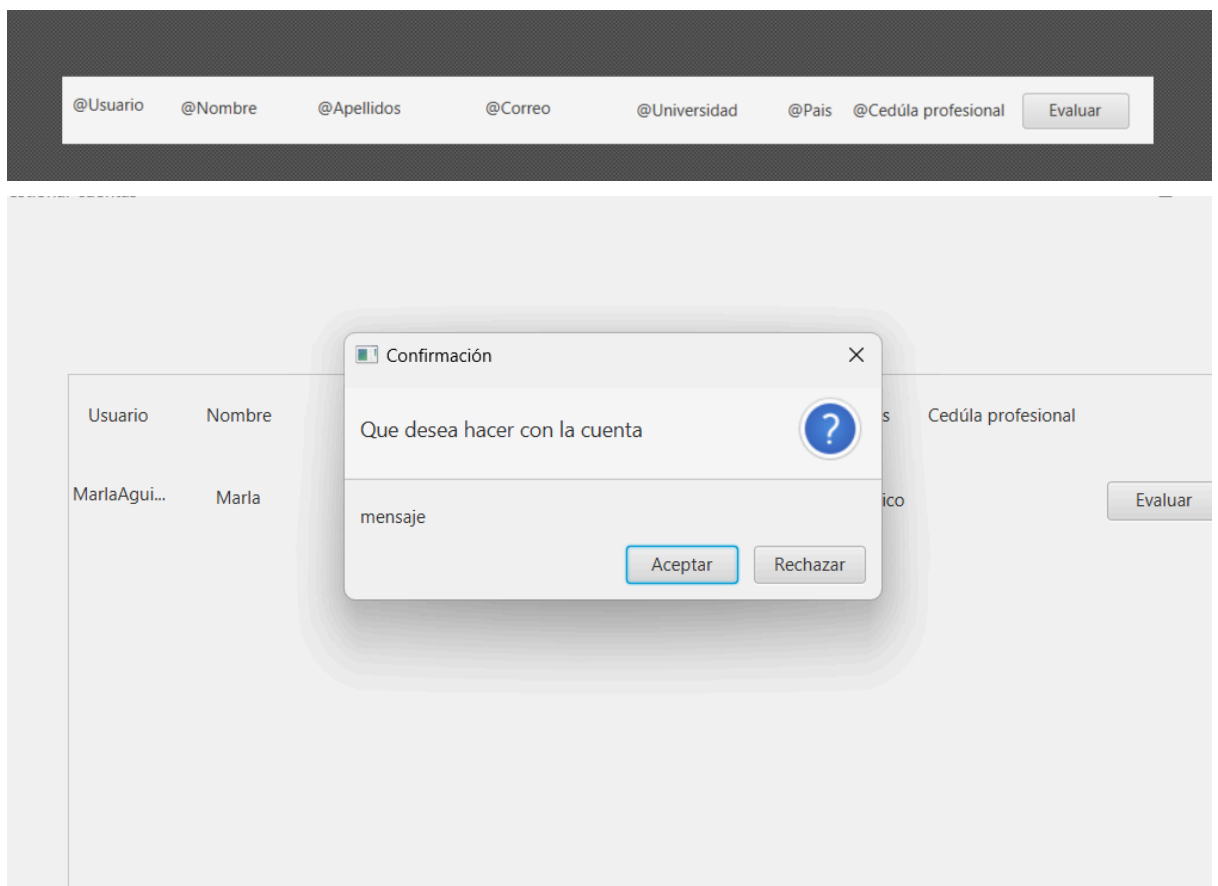
```

@Override
public void initialize (URL url, ResourceBundle resourceBundle) {
    List<Cuenta> cuentasPendientes = getCuentaEnEstadoPendiente();
    for (int i = 0; i < cuentasPendientes.size(); i++) {
        final Cuenta cuenta = cuentasPendientes.get(i);
        FXMLLoader fxmlLoader = new FXMLLoader();
        fxmlLoader.setLocation(getClass().getResource("name: ../Plantilla/CuentaItem.fxml"));
        try {
            VBox vbox = fxmlLoader.load();
            CuentaItemControlador cuentaItemController = fxmlLoader.getController();
            cuentaItemController.setCuentaObtenida(cuenta);
            cuentaItemController.setLabel(cuenta);
            cuentaItemController.getBtEvaluar()
                .setOnAction(event -> {
                    Cuenta cuentaSeleccionada = cuentaItemController.getCuentaObtenida();
                    cambiarEstadoCuenta(cuentaSeleccionada);
                    lyInformacionCuenta.getChildren().remove(vbox);
                });
        }
    }
}

```

(Esta parte podría refactorizarlo)

4. Ventana: cuentaItemControlador.



Usuario	Nombre	Apellidos	Correo electrónico	Universidad	Pais	Cedúla profesional

Al darle al botón “Aceptar” o “Rechazar” se borra ese elemento y se cambia su estado en la base de datos.

5. Se agregó un metodo extra a la clase UniversidadDB, al DAOUniversidad y al IUniversidad (Metodo getPorId)

Emmanuel Pale Molina

- Ventana: editar universidad
 - EditarUniversidad.FXML
 - EditarUniversidadControlador.java
- Se resolvieron problemas en la ventana Registrar Universidad.
 - RegistroUniversidadControlador.java
- Se hicieron cambios en el DAOUniversidad y se corrigieron los test afectados por los cambios.
 - DAOUniversidad.java
 - DAOUniversidadTest.java

Configuración de universidad

Nombre

Universidad Veracruzana

País

México

Guardar cambios

Cancelar

Configuración de universidad

Mensaje



Se han guardado los cambios exitosamente

Aceptar

País

México

Guardar cambios

Cancelar

Configuración de universidad

Confirmación

No se guardarán los cambios

Aceptar Cancelar

País

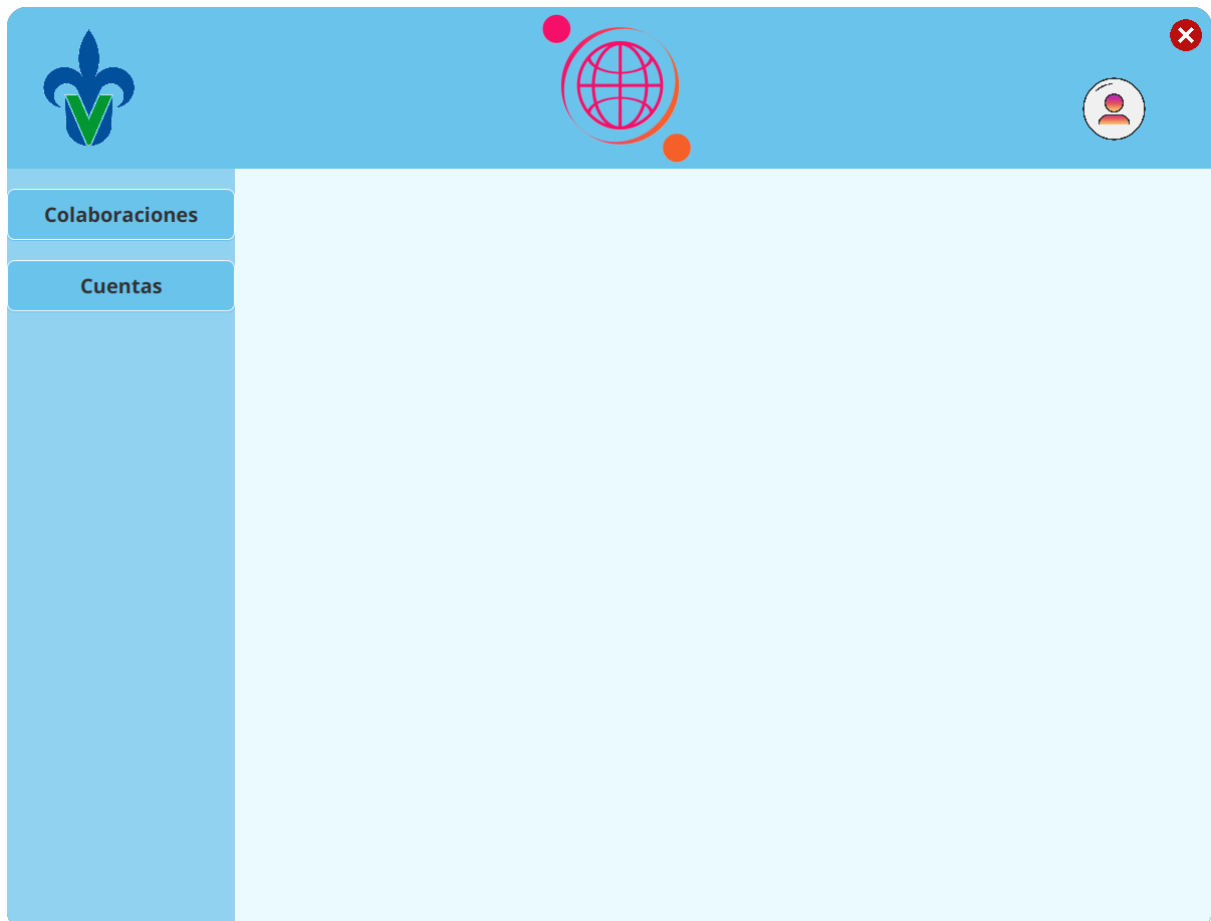
México

Guardar cambios

Cancelar

David Carrión Romero

- ventana: VentanaPrincipalAcademico



- controlador: VentanaPrincipalAcademico

```

public class VentanaPrincipalAcademicoControlador extends Application {
    2 usages
    private final Logger BITACORA = Logger.getLogger(VentanaPrincipalAcademicoControlador.class);

    1 usage
    @FXML
    private Button btnColaboraciones = new Button();

    new *
    public static void main (String[] args) { launch(args); }

    new *
    @Override
    public void start (Stage stage) {
        BorderPane root = null;

        try {
            // crear panel de fxml
            root = FXMLLoader.load(getClass().getResource("name: \"../Plantilla/VentanaPrincipalAcademico.fxml\""));
        }
        catch (IOException e) {
            BITACORA.error(e);
        }

        if (root != null) {
            stage.initStyle(StageStyle.TRANSPARENT);
            stage.setTitle("Crear actividad nueva");

            Scene escena = new Scene(root, Color.TRANSPARENT);
            escena.getStylesheets().add("InterfazGrafica/Estilos/ventana.css");

            stage.setScene(escena);
            stage.show();
        } else {
            BITACORA.error("Ocurrió un error al iniciar la ventana windowNuevaActividad");
        }
    }

    no usages new *
    public void cerrarVentana () {
        Stage window = (Stage) btnColaboraciones.getScene().getWindow();
        window.close();
    }
}

```

- estilo de ventanas css

```

.root {
    -fx-background-color: #EBFCFF;
    -fx-background-radius: 20;
}

.button-principal {
    -fx-font: bold 14px "System";
    -fx-background-color: #FA790F;
}

.button-secundario {
    -fx-font: 14px "System";
    -fx-background-color: #E7C736;
}

.barra-titulo {
    -fx-background-color: #6CC3EF;
    -fx-background-radius: 20 20 0 0;
}

.barra-secundaria {
    -fx-background-color: #91D2F3;
    -fx-background-radius: 0 0 0 20;
}

.button-menu-lateral {
    -fx-font: bold 20px "System";
    -fx-background-color: #6CC3EF;
    -fx-background-radius: 7;
    -fx-border-color: white;
    -fx-border-radius: 7;
    -fx-padding: 10px;
}

.button-menu-lateral:hover {
    -fx-background-color: #B7DCF5;
}

```

- Refactorización métodos de controlador nueva actividad

```

2 usages  ± guashasha
public void cerrarVentana () {
    Stage window = (Stage) btnCancelar.getScene().getWindow();
    window.close();
}

2 usages  ± david carrion romero +1
private static Alert crearAlerta (ErrorDAO error) {
    Alert errorAlert = new Alert(Alert.AlertType.ERROR);

    switch (error.getTipo()) {
        case VALIDACION:
            errorAlert.setHeaderText("Error de datos");
            errorAlert.setContentText("Los datos de la actividad son incorrectos, verifiquelos e intente de nuevo");
            break;

        case CONSULTA:
            errorAlert.setHeaderText("Error al agregar");
            errorAlert.setContentText("Ocurrió un error al agregar la actividad");
            break;

        case DUPLICIDAD:
            errorAlert.setHeaderText("La actividad ya existe");
            errorAlert.setContentText("Ya existe una actividad con estos datos, intente de nuevo");
            break;

        case CONEXION:
            errorAlert.setHeaderText("Error de base de datos");
            errorAlert.setContentText("Ocurrió un error al conectarse a la base de datos, intente nuevamente más tarde");
            break;
    }

    return errorAlert;
}

```

- nuevo DAO: cronograma de actividades

```

public class DAOCronogramaActividades implements IDAO<ActividadVinculada, Integer> {
    5 usages
    private static final Logger BITACORA = Logger.getLogger(DAOCronogramaActividades.class);

    ± david carrion romero +1
    @Override
    public int agregar (ActividadVinculada actividadVinculada) throws ErrorDAO {
        if (!actividadVinculada.getPeriodo()
            .esCorrecto()) {
            throw new ErrorDAO( mensaje: "El periodo especificado es incorrecto.", ErrorDAO.Tipo.VALIDACION);
        } else if (!actividadVinculada.getActividad()
            .esCorrecta()) {
            throw new ErrorDAO( mensaje: "La actividad es incorrecta", ErrorDAO.Tipo.VALIDACION);
        } else if (!actividadVinculada.getColaboracion()
            .validaNulos()) {
            throw new ErrorDAO( mensaje: "La colaboración es incorrecta", ErrorDAO.Tipo.VALIDACION);
        }

        int resultado = -1;

        try {
            resultado = CronogramaActividadDB.agregar(actividadVinculada.getActividad(), actividadVinculada.getColaboracion(), actividadVinculada.getPeriodo());
        } catch (SQLException error) {
            BITACORA.error(error);
            throw new ErrorDAO( mensaje: "Error de sql: " + error.getMessage(), ErrorDAO.Tipo.CONEXION);
        }

        if (resultado < -1) {
            throw new ErrorDAO( mensaje: "No se pudo registrar la actividad al cronograma", ErrorDAO.Tipo.INSERCIÓN);
        }

        return 1;
    }

    ± david carrion romero
    @Override
    public int modificar (ActividadVinculada actividadVinculada) throws ExecutionControl.NotImplementedException {...}

    ± david carrion romero
    @Override
    public Optional<ActividadVinculada> getPorId (Integer id) throws ErrorDAO {...}
}

```